5

159.302 CSPs

Other Strategies

Constraint Satisfaction Problems

n.h.reyes@massey.ac.nz

Reference of contents: ***MIT OpenCourseWare***

Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed **ordering** of **variables** & **values**, (e.g. random order or place variables with constraints first.)

You can usually do better by choosing an order dynamically as the search proceeds.

Ordering of variables can *have a substantial effect on the cost of finding the answer*. We can re-order variables based on information available during a search.



Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed ordering of variables & values, e.g. random order or place variables with constraints first.

You can usually do better by choosing an order dynamically as the search proceeds.

- **Most constrained variable**
  when doing forward-checking, pick variable with fewest legal values to assign next (minimise branching factor)



Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed ordering of variables & values, e.g. random order or place variables with constraints first.

You can usually do better by choosing an order dynamically as the search proceeds.

- **Most constrained variable**
  when doing forward-checking, pick variable with fewest legal values to assign next (minimise branching factor)

- **Least constraining value**
  choose value that rules out the fewest values from neighboring domains

Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed ordering of variables & values, e.g. random order or place variables with constraints first.

You can usually do better by choosing an order dynamically as the search proceeds.

- **Most constrained variable**
  when doing forward-checking, pick variable with fewest legal values to assign next (minimise branching factor)

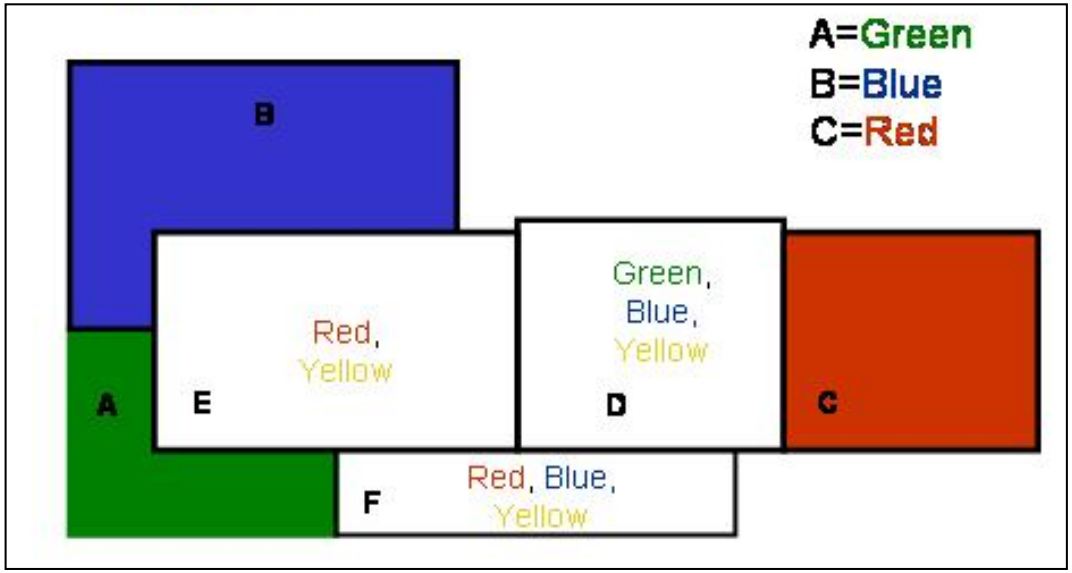
- **Least constraining value**
  choose value that rules out the fewest values from neighboring domains

e.g. This combination improves feasible N-Queens performance from about n=30 with just FC to about n=1000 with FC & ordering

Solving CSPs

Dynamic Variable and Value Ordering

Colours: R, G, B, Y



Which country should we colour next?

What colour should we pick for it?

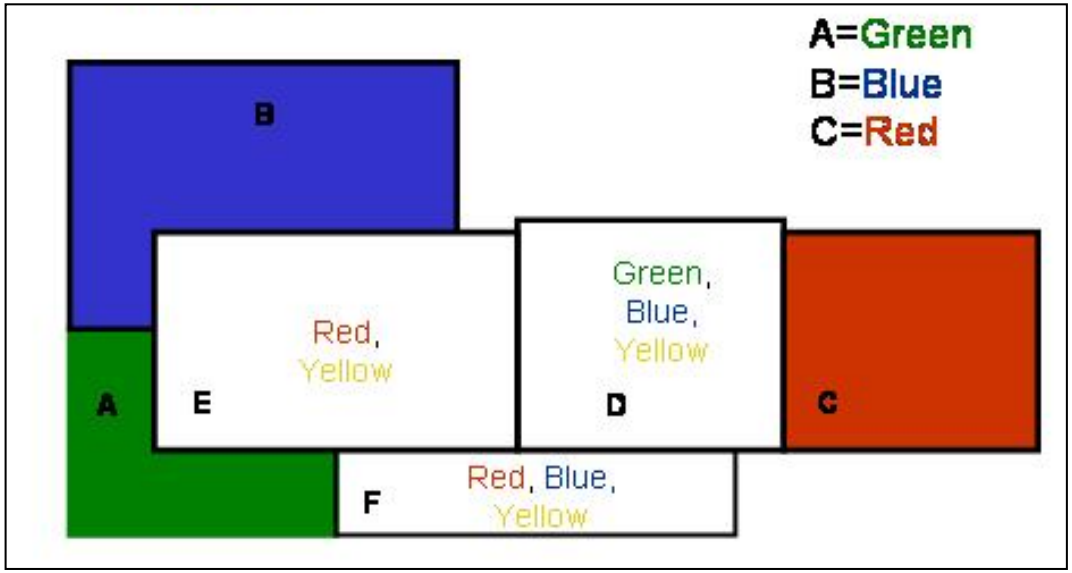
The 4-Colour Map-Colouring Problem illustrates a simple situation for variable and value ordering.



Solving CSPs

Dynamic Variable and Value Ordering

Colours: R, G, B, Y



The 4-Colour Map-Colouring Problem illustrates a simple situation for variable and value ordering.



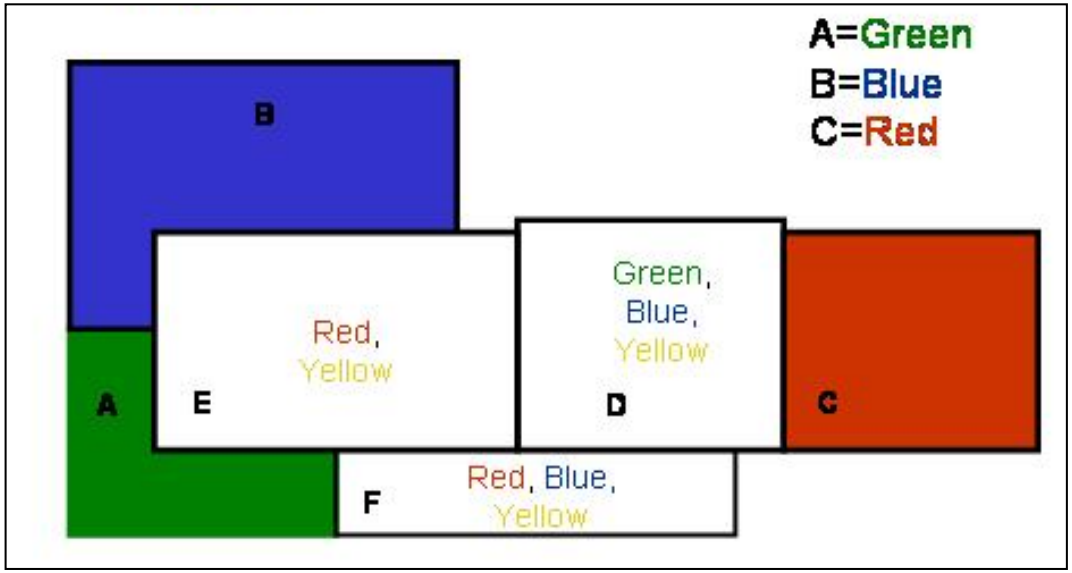
Which country should we colour next? ⇒ **E** is most constrained variable (smallest domain)

What colour should we pick for it?

Solving CSPs

Dynamic Variable and Value Ordering

Colours: R, G, B, Y



The 4-Colour Map-Colouring Problem illustrates a simple situation for variable and value ordering.



Which country should we colour next? → **E** is most constrained variable (smallest domain)

What colour should we pick for it? → **Red –** least constraining value (eliminates fewest values from neighboring domains)

Solving CSPs

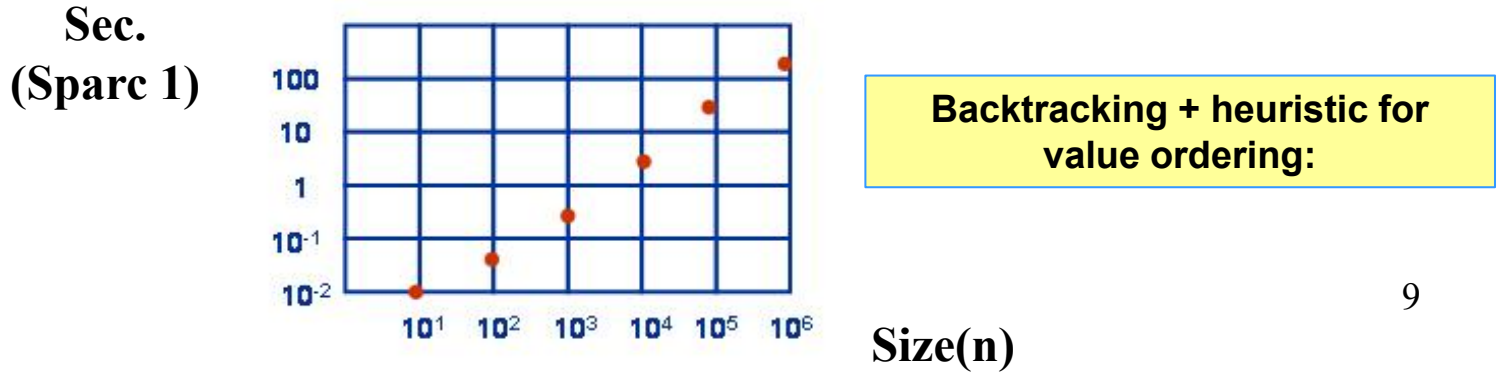
Incremental Repair (Min-Conflict Heuristic)

1. Initialise a candidate solution using "local greedy" heuristic – get solution "near" correct one.

2. Select a variable in conflict and assign it a value that minimises the number of conflicts (break ties randomly).

- Can use this heuristic as **part of systematic backtracker** that uses heuristics to do **value ordering** or in a local hill-climber (without backup).

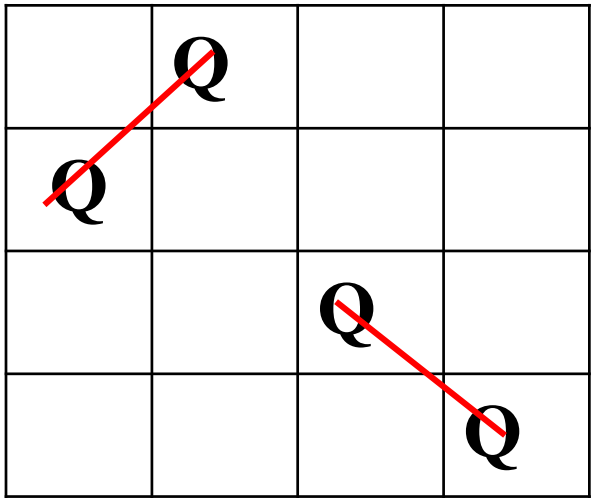
Performance on N-Queens (with ***good*** initial guess)



Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

1. Initialise a candidate solution using "greedy" heuristic – get solution "near" correct one.

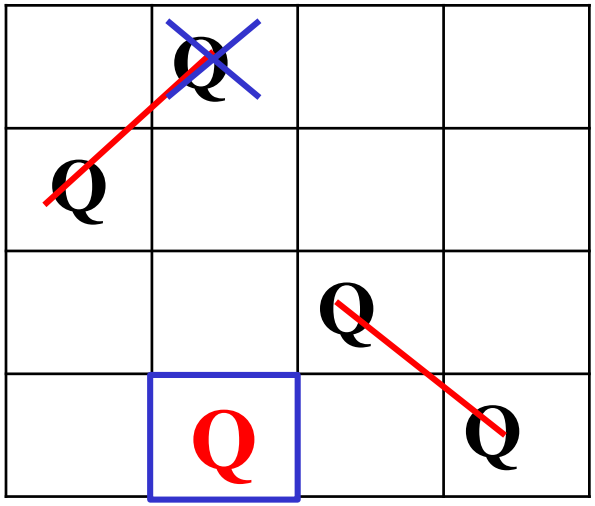


number of variables in conflict=4

Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

2. Select a variable in conflict and assign it a value that minimises the number of conflicts (break ties randomly).

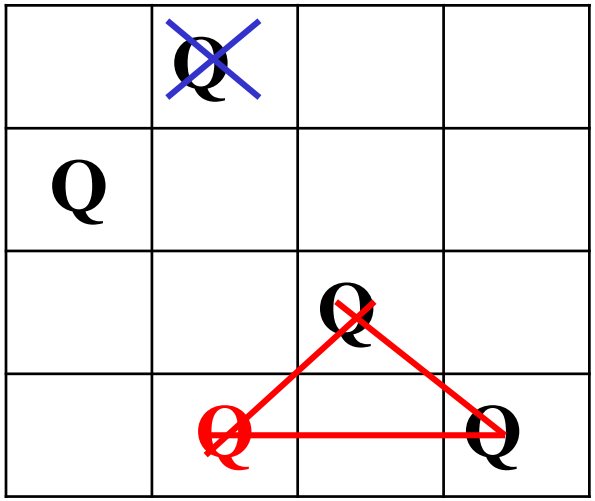


number of variables in conflict=3

Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

2. Select a variable in conflict and assign it a value that minimises the number of conflicts (break ties randomly).

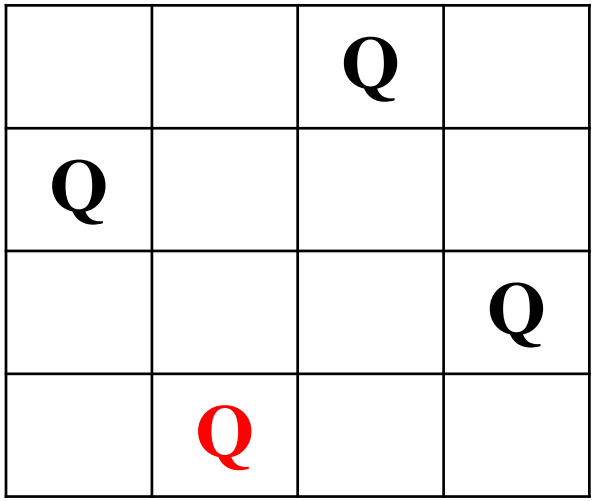


number of variables in conflict=3

Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

After many trials, a solution may be found. Otherwise, re-initialise the system with another random state.



Conflicts=0

Solving CSPs

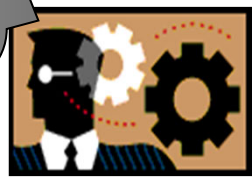
Min-Conflict Heuristic

The pure hill climber (without backtracking) can get stuck in local minima. Can add random moves to attempt getting out of minima – generally quite effective. Can also use weights on violated constraints & increase weight every cycle if it remains violated.

GSAT

- Restart the search with a new random initial state.
- Randomised hill-climber used to solve SAT problems. One of the most effective methods ever found for this problem.

GSAT can solve SAT problems of mind-boggling complexity. It has set a new standard for classifying SAT problems as "*hard*", because almost any random problem is "easy" for GSAT.



Solving CSPs

Min-Conflict Heuristic

GSAT

Example

Clauses

$A \vee \neg B \vee C$

$\neg A \vee C \vee D$

$B \vee D \vee \neg E$

$\neg C \vee \neg D \vee \neg E$

$\neg A \vee \neg C \vee E$

AND

Clauses are combined together using an AND logical connective.

Solving CSPs

GSAT as Heuristic Search

State Space: Space of all full assignments to variables

Initial State: a random full assignment

Goal State: a satisfying assignment

Actions: flip value of one variable in current assignment

Heuristic: the number of satisfied clauses (constraints); we want to maximise this score. Alternatively, minimise the number of unsatisfied clauses (constraints).

Solving CSPs

Algorithm: GSAT(F)

- **For i=1 to MaxTries**
  - Select a complete random assignment **A**
  - *Score* = number of satisfied clauses
  - **For i=1 to MaxFlips**
    - If (**A** satisfies all clauses in **F**) {
      return **A**
      }
    - Else {
      Flip a variable that maximises the *Score*
      }
    - Flip a randomly chosen variable if no variable flip increases the *Score*

MaxTries and **MaxFlips** are user-defined. These guard against **local minima** in the search.

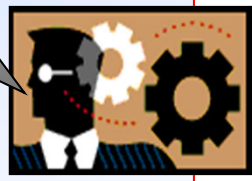


Solving CSPs

Algorithm: WALKSAT(F)

- **For i=1 to MaxTries**
  - Select a complete random assignment **A**
  - *Score* = number of satisfied clauses
  - **For i=1 to MaxFlips**
    - If (**A** satisfies all clauses in **F**) {
      return **A**
      }
    - Else {
      - **With probability p** //**GSAT**
        - Flip a variable that maximises the *Score*
        - Flip a randomly chosen variable if no variable flip increases the Score
      - **With probability (1-p)** //**Random Walk**
        - Pick a random unsatisfied clause C
        - Flip a randomly chosen variable in C
      }

It turns out that adding **more randomness** is a more effective strategy!



https://www.cs.rochester.edu/u/kautz/walksat/